부록 · 실습 트러블슈팅

컨테이너·포트 — 만들기·켜기·연결

도커 실습에서 가장 자주 만나는 오류입니다. 오류 문구로 찾으세요.

No such container: de-mysql

어디서: 2-1 실습점검 4단계 · 2-2 시작 · 2-5 실습점검 (이번 수업 중 수강생 질문 — 2-5 에서는 2-1 실습점검 5단계 「전체 재설정」을 쳐서 볼륨까지 지워진 경우였습니다)

원인: 그 이름의 컨테이너가 아직 없거나(만들기 전에 exec·start 를 친 것) 지운 것입니다. 틀린 것이 아니라 만들기 전이라는 뜻입니다. 데이터(볼륨 de-mysql-data)까지 지워졌는지에 따라 복구가 갈립니다.

해결: 볼륨부터 확인합니다. 있으면 2-1 기동 명령 한 줄로 컨테이너만 다시 만들면 표가 전부 그대로이고, 없으면 같은 명령으로 빈 shop 이 생깁니다 — 처음 만들 때는 logs 에 port: 0 이 먼저 찍히고 10~20초 뒤 port: 3306 이 나옵니다. 빈 shop 의 SHOW TABLES 가 Empty set 인 것은 정상이며, 그 자리에서 그날 실습의 CREATE TABLE 부터 이어가고 앞 차시 표는 수업 뒤 노트대로 다시 넣습니다.

```bash
docker volume ls --filter name=de-mysql-data
docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 330
6:3306 -v de-mysql-data:/var/lib/mysql mysql:9.7
docker logs --tail 5 de-mysql      # 끝 줄 port: 3306 까지
docker exec -it -e LANG=C.UTF-8 de-mysql mysql -uroot -pdataeng123 shop
```

Invalid MySQL server upgrade: Cannot upgrade from 80046 to 90704

어디서: 2-1 기동과 접속 1단계 · 2-5 실습점검 — 8.0 으로 만든 de-mysql-data 에 mysql:9.7 을 붙일 때 (2026-10-24 실측 재현)

원인: 노트가 MySQL 8.0 에서 9.7 로 바뀌기 전에 만든 볼륨에는 8.0 데이터가 들어 있습니다. 9.7 은 바로 앞 장기 지원판(8.4)의 데이터만 받아서, 8.0 데이터를 보면 시작하자마자 멈춥니다 — docker ps -a 에 Exited (1), docker logs 에 이 문구와 Data Dictionary initialization failed 가 찍힙니다. 볼륨은 망가지지 않습니다(8.0 으로 다시 열면 그대로 열림).

해결: 멈춘 컨테이너를 지운 뒤 하나를 고릅니다. ① 데이터를 그대로 쓰려면 이미지만 mysql:8.0 으로 — 노트의 명령과 결과는 8.0 에서도 같습니다. ② 새로 시작하려면 볼륨을 지우거나 -v 에 새 볼륨 이름을 줍니다. ③ 데이터를 9.7 로 옮기려면 mysql:8.4 로 한 번 띄워 끝 줄에 port: 3306 이 나온 것을 본 뒤(그동안 8.0 → 8.4 로 고쳐 씁니다) docker stop de-mysql 로 멈추고 지운 다음 mysql:9.7 로 띄웁니다(실측 — 8.4.13 을 거쳐 9.7.4 에서 표가 그대로 열림).

```bash
docker rm -f de-mysql

# ① 8.0 그대로
docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 330
6:3306 -v de-mysql-data:/var/lib/mysql mysql:8.0

# ② 새로 — 볼륨을 지우고(데이터가 지워집니다) 9.7 로
# docker volume rm de-mysql-data
# docker run -d --name de-mysql -e MYSQL_ROOT_PASSWORD=dataeng123 -e MYSQL_DATABASE=shop -p 33
06:3306 -v de-mysql-data:/var/lib/mysql mysql:9.7
```

The container name "/web" is already in use (Conflict)

어디서: 1-3 컨테이너 기초 · 2-1 포트를 바꿔 다시 띄울 때

원인: 같은 이름의 컨테이너가 멈춘(exited) 채로 남아 있습니다. 멈춘 컨테이너도 이름을 붙들고 있습니다.

해결: 확인하고 지운 뒤 다시 만듭니다. 데이터는 볼륨에 있으면 그대로 남습니다.

```bash
docker ps -a
docker rm 이름          # 돌고 있으면 docker rm -f 이름
```

port is already allocated · Ports are not available

어디서: 1주차 · 2-1 · 2-2 · 4주차 MinIO · 5주차 Spark · 7주차 · 8주차 Airflow

원인: 그 포트를 다른 컨테이너(앞 차시에 띄운 것)나 내 컴퓨터에 설치된 프로그램이 이미 쓰고 있습니다.

해결: 누가 쓰는지 보고 멈추거나, 내 쪽 포트 번호만 바꿉니다(-p 3307:3306 처럼 앞 숫자).

```bash
docker ps                                    # 앞 차시 컨테이너가 떠 있는지
lsof -nP -iTCP:3306 -sTCP:LISTEN             # macOS·리눅스 — 포트 번호만 바꿔 씁니다
```

Windows PowerShell 은 netstat -ano | findstr ":3306" 으로 봅니다. Windows 는 재부팅하면 멈춰 둔 MySQL80 서비스가 다시 켜질 수 있습니다(2-2).

켜자마자 접속하면 실패한다

어디서: 2-1 MySQL · 2-3 MongoDB · 6주차 Kafka

원인: 컨테이너가 Up 이어도 안의 서버는 아직 준비 중입니다. 준비 완료 표시를 보고 접속합니다.

해결: 서버마다 준비 완료 표시가 다릅니다.

```bash
# MySQL — 끝에 port: 3306 이 붙은 ready for connections 줄 (처음엔 port: 0 이 먼저 한 번 찍힙니다 — 아직 아님)
docker logs --tail 5 de-mysql

# MongoDB — 로그 문구 대신 실제로 접속해 봅니다. 1 이 나오면 준비 완료
docker exec de-mongo mongosh --quiet --eval "db.runCommand({ ping: 1 }).ok"

# Kafka — "Kafka Server started" 줄
docker logs kafka
```

MongoDB 의 「Waiting for connections」를 tail 3줄에서 찾는 안내가 많은데, 이 버전은 그 뒤에 줄이 더 붙어 보이지 않습니다(2-3 실측).

Unknown MySQL server host 'host.docker.internal' (ERROR 2005)

어디서: 2-1 기동과 접속 4단계 (이번 수업 중 수강생 질문)

원인: host.docker.internal 은 컨테이너에서 내 컴퓨터를 가리키는 이름입니다. Docker Desktop(macOS·Windows)은 자동으로 넣어 주지만, 리눅스나 WSL2 우분투에 도커 엔진만 설치한 경우에는 없습니다. 설정이 빠진 것이 아닙니다.

해결: --add-host 로 직접 넣어 줍니다. Docker Desktop 에서는 붙여도 결과가 같으니 항상 붙여도 됩니다.

```bash
docker run --rm --add-host=host.docker.internal:host-gateway mysql:9.7 \
  mysql -h host.docker.internal -P 3306 -uroot -p -e "SELECT VERSION();"
```

pull access denied for minio/minio, repository does not exist

어디서: 4주차 MinIO (2026-10-02 실측 — 수업 전 발견)

원인: MinIO 공식 이미지 minio/minio · minio/mc 를 Docker Hub 에서 더 이상 내려받을 수 없습니다(404).

해결: 같은 MinIO 를 빌드한 pgsty/minio · pgsty/mc 로 이름만 바꿉니다. 과정 노트는 이미 바꿔 두었습니다. 옛 자료를 따라 하다 이 오류가 나면 compose 의 image 줄을 확인합니다.

```bash
docker pull pgsty/minio
docker pull pgsty/mc
```

grep: command not found · `sh` is not a recognized command (mc 컨테이너)

어디서: 4~6주차 mc 명령

원인: mc 이미지에는 grep·awk·sed·wget·curl 이 없고, 이미지가 mc 명령으로 시작합니다(2026-10-03 실측).

해결: 거르기(grep 등)는 컨테이너 밖에서 하고, docker run 으로 셸을 쓰려면 --entrypoint sh 를 줍니다.

```bash
docker compose exec -T mc mc ls --recursive local/warehouse | grep -c .parquet
docker run --rm --network week6_default --entrypoint sh pgsty/mc -c "mc alias set m http://min
io:9000 minioadmin minioadmin && mc ls m"
```

컨테이너 안에서 localhost 로 붙으면 Connection refused · UnknownHostException

어디서: 5주차 Spark ↔ MinIO

원인: 컨테이너 안에서 localhost 는 그 컨테이너 자신입니다. 다른 컨테이너는 같은 compose 네트워크 안에서 서비스 이름으로 찾습니다.

해결: 서비스들을 한 docker-compose.yml 에 두고, 주소를 서비스 이름으로 적습니다(예: http://minio:9000). 내 컴퓨터(호스트)에서 붙을 때만 localhost 를 씁니다.

데이터 엔지니어링 & RAG 파이프라인 · 부록 — 서버 주소(SERVER_IP)·계정(devuser)·비밀번호(CHANGE_ME…)는 자리표시입니다. 본인 환경의 값으로 바꿔 쓰세요.